Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

Практична робота №3

з дисципліни «Розробка проектів засобами платформи .NET»

Тема: «Узагальнення, делегати, події та лямбда-вирази»

Виконав: студент гр. 2ПІ-24б
Слободян І. В.

Перевірив:
Позур Михайло Юрійович

м. Вінниця - 2026

GitHub репозиторій: https://github.com/2005igor2023-crypto/dotnetvntu

1. Мета роботи

Навчитися створювати власні узагальнені (generic) класи та методи з обмеженнями типів, передавати логіку як параметри за допомогою делегатів (Func, Action), проектувати події за стандартом .NET з використанням власного нащадка EventArgs, а також застосовувати лямбда-вирази для обробки подій, критеріїв фільтрації та реалізації анонімних функцій.

2. Завдання варіанта №1

• **Предметна область:** Кеш курсів валют.

• **Структура:** Створити узагальнений клас Cache<TKey, TValue> where TKey : notnull, місткість якого задається у конструкторі (для наочності обрано місткість 3). Значення зберігаються у словнику Dictionary<TKey, TValue>, а черговість додавання ключів для визначення найстарішого запису - у списку List<TKey>.

• **Методи кешу:**

• TValue GetOrAdd(TKey key, Func<TKey, TValue> load, out bool fromCache) - перевіряє наявність значення за ключем. Якщо ключ є в кеші, повертає значення з кешу; якщо немає - викликає функцію load(key), зберігає результат і повертає його. Якщо кеш заповнений, перед додаванням нового елемента видаляється найстаріший запис.

• void ForEach(Action<TKey, TValue> action) - обходить усі збережені елементи кешу у порядку їх додавання та виконує задану дію.

• void Clear() - очищає вміст кешу.

• **Імітація сервера банку:** Метод decimal LoadRate(string code), що імітує звернення до зовнішнього API: виводить сповіщення про запит до сервера та повертає курс із заздалегідь заповненого словника (USD, EUR, PLN, GBP, CHF, CAD). Цей метод передається як аргумент-делегат у GetOrAdd.

• **Подія:** Подія ItemRemoved з аргументами ItemRemovedEventArgs<TKey, TValue> (ключ, значення, дата та час витіснення), яка виникає під час автоматичного видалення найстарішого запису через переповнення місткості.

• Підписники на подію:

1. Іменований метод OnItemRemovedNamedMethod для виведення інформаційного повідомлення у консоль.

2. Лямбда-вираз для інкременту лічильника витіснених записів та виведення поточної кількості видалень.

• **Використання лямбда-виразів:** Використано щонайменше у 3 місцях коду (підписник на подію, фільтрація за порогом курсу через замикання, завантаження у другому екземплярі кешу).

• **Другий екземпляр узагальненого кешу:** Демонстрація Cache<int, string> для кешування імен користувачів за числовим ідентифікатором із завантаженням через лямбду id => $"Користувач #{id}...".

• **Інтерактивне меню:** Можливість додавання та перегляду курсів валют, фільтрації, перегляду другого кешу та виходу.

3. Опис реалізованої архітектури та структури проєкту

Проєкт структуровано відповідно до принципів модульності:

1. Events/ItemRemovedEventArgs.cs: Узагальнений клас аргументів події, успадкований від EventArgs. Містить властивості Key, Value та RemovedAt.

2. Collections/Cache.cs: Клас Cache<TKey, TValue> з обмеженням where TKey : notnull. Інкапсулює колекції Dictionary<TKey, TValue> та List<TKey>. Реалізує методи GetOrAdd, ForEach, Clear та подію ItemRemoved. Виклик події здійснюється через захищений віртуальний метод OnItemRemoved за допомогою оператора безпечного виклику ?.Invoke(this, e).

3. Program.cs: Головний модуль із точкою входу Main, базою даних банку BankDatabase, методом завантаження курсів LoadRate, обробниками подій та текстовим меню.

4. Лістинг програмного коду

4.1. Файл Events/ItemRemovedEventArgs.cs

```
using System;

namespace GenericsDelegatesEvents.Events;

public class ItemRemovedEventArgs<TKey, TValue> : EventArgs
{
    public TKey Key { get; }
    public TValue Value { get; }
    public DateTime RemovedAt { get; }

    public ItemRemovedEventArgs(TKey key, TValue value)
    {
        Key = key;
        Value = value;
        RemovedAt = DateTime.Now;
    }
}
```

4.2. Файл Collections/Cache.cs

```
using System;
using System.Collections.Generic;
using GenericsDelegatesEvents.Events;

namespace GenericsDelegatesEvents.Collections;

public class Cache<TKey, TValue> where TKey : notnull
{
    private readonly int _capacity;
    private readonly Dictionary<TKey, TValue> _storage;
    private readonly List<TKey> _keysOrder;

    public event EventHandler<ItemRemovedEventArgs<TKey, TValue>>? ItemRemoved;

    public int Capacity => _capacity;
    public int Count => _storage.Count;

    public Cache(int capacity)
    {
        if (capacity <= 0)
        {
            throw new ArgumentOutOfRangeException(nameof(capacity), "Місткість кешу має бути
більшою за нуль.");
        }

        _capacity = capacity;
```

```
    _storage = new Dictionary<TKey, TValue>(capacity);
    _keysOrder = new List<TKey>(capacity);
}

public TValue GetOrAdd(TKey key, Func<TKey, TValue> load, out bool fromCache)
{
    ArgumentNullException.ThrowIfNull(key);
    ArgumentNullException.ThrowIfNull(load);

    if (_storage.TryGetValue(key, out TValue? cachedValue))
    {
        fromCache = true;
        return cachedValue;
    }

    fromCache = false;
    TValue loadedValue = load(key);

    if (_storage.Count >= _capacity)
    {
        EvictOldest();
    }

    _storage[key] = loadedValue;
    _keysOrder.Add(key);

    return loadedValue;
}

public TValue GetOrAdd(TKey key, Func<TKey, TValue> load)
{
    return GetOrAdd(key, load, out _);
}

public void ForEach(Action<TKey, TValue> action)
{
    ArgumentNullException.ThrowIfNull(action);

    foreach (TKey key in _keysOrder)
    {
        if (_storage.TryGetValue(key, out TValue? value))
        {
            action(key, value);
        }
    }
}

public bool ContainsKey(TKey key)
{
    ArgumentNullException.ThrowIfNull(key);
    return _storage.ContainsKey(key);
}

public void Clear()
{
    _storage.Clear();
    _keysOrder.Clear();
```

```
    }

    private void EvictOldest()
    {
        if (_keysOrder.Count == 0) return;

        TKey oldestKey = _keysOrder[0];
        _keysOrder.RemoveAt(0);

        if (_storage.Remove(oldestKey, out TValue? removedValue))
        {
            OnItemRemoved(new ItemRemovedEventArgs<TKey, TValue>(oldestKey, removedValue));
        }
    }

    protected virtual void OnItemRemoved(ItemRemovedEventArgs<TKey, TValue> e)
    {
        ItemRemoved?.Invoke(this, e);
    }
}
```

4.3. Файл Program.cs

```
using System;
using System.Collections.Generic;
using System.Text;
using GenericsDelegatesEvents.Collections;
using GenericsDelegatesEvents.Events;

namespace GenericsDelegatesEvents;

internal class Program
{
    private static int _removedItemsCount = 0;

    private static readonly Dictionary<string, decimal> BankDatabase =
new(StringComparer.OrdinalIgnoreCase)
    {
        ["USD"] = 41.50m,
        ["EUR"] = 44.80m,
        ["PLN"] = 10.30m,
        ["GBP"] = 52.40m,
        ["CHF"] = 46.20m,
        ["CAD"] = 30.15m
    };

    private static void Main()
    {
        Console.OutputEncoding = Encoding.UTF8;
        Console.InputEncoding = Encoding.UTF8;

        Cache<string, decimal> rateCache = new(capacity: 3);

        rateCache.ItemRemoved += OnItemRemovedNamedMethod;
        rateCache.ItemRemoved += (sender, e) =>
        {
            _removedItemsCount++;
```

```
        Console.WriteLine($"[Лямбда-підписник] Загальна кількість витіснених записів:
{_removedItemsCount}");
    };

    rateCache.GetOrAdd("USD", LoadRate);
    rateCache.GetOrAdd("EUR", LoadRate);

    bool isRunning = true;
    while (isRunning)
    {
        PrintMenu(rateCache);
        int choice = ReadInt("Ваш вибір: ", 0, 5);
        Console.WriteLine();

        isRunning = choice switch
        {
            1 => HandleGetRate(rateCache),
            2 => HandleShowCache(rateCache),
            3 => HandleFilterByThreshold(rateCache),
            4 => HandleSecondCacheDemo(),
            5 => HandleClearCache(rateCache),
            0 => false,
            _ => true
        };

        if (isRunning)
        {
            Console.WriteLine("\nНатисніть Enter для повернення в меню...");
            Console.ReadLine();
        }
    }
}

private static void PrintMenu(Cache<string, decimal> cache)
{

Console.WriteLine("\n============================================================");
    Console.WriteLine("    Практична робота №3 | Варіант 1                  ");
    Console.WriteLine("         Кеш курсів валют (Cache<TKey, TValue>)       ");

Console.WriteLine("============================================================");
    Console.WriteLine($"Поточний стан кешу: {cache.Count} / {cache.Capacity} записів");
    Console.WriteLine("1. Отримати курс валюти (USD, EUR, PLN, GBP, CHF, CAD)");
    Console.WriteLine("2. Показати поточний вміст кешу");
    Console.WriteLine("3. Показати валюти з курсом, вищим за поріг (делегат Action +
лямбда)");
    Console.WriteLine("4. Демонстрація другого узагальненого кешу Cache<int, string>");
    Console.WriteLine("5. Очистити кеш валют");
    Console.WriteLine("0. Вихід");
    Console.WriteLine("------------------------------------------------------------");
}

public static decimal LoadRate(string code)
{
    Console.WriteLine($"[Сервер банку] Виконується запит курсу для валюти {code}...");
    if (BankDatabase.TryGetValue(code, out decimal rate))
    {
```

```
            return rate;
        }
        throw new KeyNotFoundException($"Валюта '{code}' не знайдена в базі банку.");
    }

    private static void OnItemRemovedNamedMethod(object? sender, ItemRemovedEventArgs<string,
decimal> e)
    {
        Console.ForegroundColor = ConsoleColor.Cyan;
        Console.WriteLine($"[Іменований підписник] Видалено з кешу найстаріший запис: {e.Key}
= {e.Value:F2} UAH (час: {e.RemovedAt:HH:mm:ss})");
        Console.ResetColor();
    }
}
```

5. Результати виконання програми

5.1. Початкове наповнення та отримання курсів валют

```
[Сервер банку] Виконується запит курсу для валюти USD...
[Сервер банку] Виконується запит курсу для валюти EUR...

============================================================
   Практична робота №3 | Варіант 1
      Кеш курсів валют (Cache<TKey, TValue>)
============================================================
Поточний стан кешу: 2 / 3 записів
1. Отримати курс валюти (USD, EUR, PLN, GBP, CHF, CAD)
2. Показати поточний вміст кешу
3. Показати валюти з курсом, вищим за поріг (делегат Action + лямбда)
4. Демонстрація другого узагальненого кешу Cache<int, string>
5. Очистити кеш валют
0. Вихід
------------------------------------------------------------
Ваш вибір: 1

--- Отримання курсу валюти ---
Введіть код валюти: USD

Результат: 1 USD = 41.50 UAH [З КЕШУ]
```

5.2. Витіснення найстарішого елемента при переповненні місткості (спрацювання події)

При додаванні четвертої валюти (PLN, а згодом GBP) ліміт місткості (3) вичерпується, найстаріший елемент видаляється, що викликає подію ItemRemoved з обома підписниками:

```
--- Отримання курсу валюти ---
Введіть код валюти: GBP
[Сервер банку] Виконується запит курсу для валюти GBP...
[Іменований підписник] Видалено з кешу найстаріший запис: USD = 41.50 UAH (час: 12:15:30)
[Лямбда-підписник] Загальна кількість витіснених записів: 1
```

Результат: 1 GBP = 52.40 UAH [ЗАВАНТАЖЕНО З СЕРВЕРА]

5.3. Фільтрація за пороговим значенням курсу через ForEach із замиканням

--- Фільтрація валют за пороговим значенням курсу ---
Введіть мінімальний поріг курсу (UAH): 42.0

Валюти у кеші з курсом >= 42.00 UAH:
• EUR : 44.80 UAH
• GBP : 52.40 UAH

5.4. Демонстрація другого узагальненого кешу Cache<int, string>

--- Демонстрація іншого типу кешу: Cache<int, string> ---
Створюємо кеш користувачів за числовим ID місткістю 2 елементи.

Запит 1: ID = 101
-> Імітація читання з БД користувача #101...
Отримано: Користувач #101 (Слободян Ігор) (з кешу: False)

Запит 2: ID = 101 (повторно)
Отримано: Користувач #101 (Слободян Ігор) (з кешу: True)

Запит 3: ID = 102
Отримано: Користувач #102 (Студент 2ПІ-24б)

Запит 4: ID = 103 (переповнення місткості, видалення найстарішого)
[Подія другого кешу] Видалено користувача Id=101: 'Користувач #101 (Слободян Ігор)'
Отримано: Користувач #103 (Новий гість)

Поточний вміст другого кешу:
ID: 102 -> Користувач #102 (Студент 2ПІ-24б)
ID: 103 -> Користувач #103 (Новий гість)

6. Відповіді на контрольні питання

1. Які проблеми колекцій на основі object (ArrayList) усувають узагальнення?

Узагальнення усувають дві головні проблеми необолонкових колекцій:

• Відсутність типобезпеки: у ArrayList можна випадково додати об'єкти несумісних типів, що призведе до InvalidCastException під час виконання програми. У generic-колекціях тип перевіряється на етапі компіляції.

• Накладні витрати на упаковку та розпаковку (boxing/unboxing): для значущих типів збереження в object вимагає виділення пам'яті в купі (heap), що суттєво уповільнює роботу та навантажує збирач сміття.

2. Чим подія (event) відрізняється від звичайного публічного поля-делегата? Які операції з подією доступні ззовні класу-власника?

Подія є інкапсульованою обгорткою над делегатом. Якщо публічне поле-делегат ззовні класу можна викликати напряму або випадково затерти всі підписки оператором =, то для event ззовні класу дозволені лише операції підписки += та відписки -=. Викликати подію та очистити список виклику може тільки сам клас-власник.

3. У чому суть замикання (closure) у лямбда-виразі? Що виведе цикл for, у якому лямбди захоплюють змінну циклу, і як це виправити?

Замикання - це механізм, за якого лямбда-вираз захоплює змінні із зовнішньої області видимості. Компілятор створює службовий прихований клас для збереження посилань на ці змінні. Якщо в циклі for лямбда захоплює змінну циклу i, то на момент виконання всі створені лямбди прочитають кінцеве значення i (наприклад, довжину масиву). Щоб це виправити, всередині тіла циклу створюють локальну копію змінної: int copy = i; і захоплюють саме її.

4. Чим відрізняються делегати Func, Action та Predicate? Наведіть приклад кожного зі свого варіанта.

- Func<T, TResult> - приймає вхідні параметри та обов'язково повертає значення (наприклад: Func<string, decimal> load для методу завантаження курсу валюти).
- Action<T> - приймає вхідні параметри, але повертає void (наприклад: Action<string, decimal> у методі ForEach для обходу кешу).
- Predicate<T> - приймає один параметр та повертає bool (використовується для перевірки відповідності умові, еквівалентний Func<T, bool>).

5. Навіщо потрібні обмеження типу-параметра (where T : ...)? Що станеться, якщо прибрати обмеження з узагальненого класу вашого варіанта?

Обмеження визначають вимоги до типів-аргументів і дозволяють компілятору знати про доступні методи та властивості типу. Обмеження where TKey : notnull у нашому класі Cache<TKey, TValue> гарантує, що ключ не набуде значення null, оскільки внутрішній Dictionary<TKey, TValue> генерує ArgumentNullException при спробі використати null як ключ.

6. Як компілятор визначає тип-аргумент під час виклику узагальненого методу без явного зазначення <T>?

Компілятор використовує механізм виведення типів (type inference): він аналізує типи фактичних аргументів, які передаються у метод під час виклику, і автоматично підставляє відповідний тип.

7. Що таке делегат? Як передача делегата як параметра дозволяє одному методу обслуговувати різні сценарії?

Делегат - це типобезпечний покажчик на метод. Передача делегата як параметра дозволяє інвертувати керування: метод описує загальний алгоритм (наприклад, перевірка кешу, видалення найстарішого елемента), а конкретну дію (джерело отримання даних, спосіб обробки елемента) визначає клієнтський код.

8. Що таке багатоадресний (multicast) делегат? Що поверне виклик multicast-делегата Func<int> з трьома методами?

Multicast-делегат зберігає список викликів із кількох методів (InvocationList). При його виклику всі методи виконуються послідовно у порядку додавання. Якщо делегат повертає значення (наприклад Func<int>), результатом виклику буде значення, повернене останнім методом у ланцюжку (значення попередніх методів втрачаються).

9. Яку сигнатуру має обробник події за стандартним шаблоном .NET і що передається в параметрах sender та e?

Стандартна сигнатура обробника події: void Handler(object? sender, TEventArgs e), де:

• sender - посилання на екземпляр об'єкта, який спричинив подію.

• e - об'єкт класу, успадкованого від EventArgs, що містить детальні дані про подію.

10. Навіщо подію спричиняють через окремий метод OnXxx і чому виклик іде через ?.Invoke?

Метод OnXxx роблять захищеним віртуальним (protected virtual), щоб похідні класи могли перевизначити логіку виклику події без порушення інкапсуляції. Виклик через ?.Invoke запобігає виникненню NullReferenceException, якщо на подію наразі не підписано жодного обробника, а також забезпечує потокобезпечність читання посилання на делегат.

11. Чому -= з лямбда-виразом, записаним прямо в операторі, не відписує обробник? Як відписати обробник-лямбду?

Кожен лямбда-вираз компілюється в окремий анонімний метод. При записі event -= (s, e) => ... створюється новий екземпляр делегата, посилання на який не збігається з раніше зареєстрованим екземпляром у списку підписників. Щоб успішно відписати лямбду, її посилання слід попередньо зберегти у змінній типу відповідного делегата та використовувати цю змінну в операторах += та -=.

12. Коли краще використати іменований метод, а коли лямбда-вираз? Наведіть приклад зі свого коду.

Іменований метод доцільно використовувати, коли логіка обробника складна, перевищує кілька рядків, вимагає самостійного повторного використання або якщо планується подальша явна відписка від події (наприклад, метод OnItemRemovedNamedMethod). Лямбда-вираз є ідеальним для коротких одноразових операцій, локальної фільтрації та захоплення змінних із контексту (наприклад, захоплення порогового значення rate >= threshold у пункті фільтрації).

7. Висновки

У ході виконання практичної роботи №3 закріплено практичні навички проектування узагальнених класів C# з обмеженнями типів на прикладі кешу з витісненням найстаріших записів. Опановано використання делегатів Func та Action, роботу з подіями за стандартом .NET, створення користувацьких класів-аргументів подій, а також застосування лямбда-виразів для локальної обробки даних та замикань.